Fast Density Peaks Clustering Algorithm Based on Approximate *k*-Nearest Neighbors

Shifei Ding, *Senior Member, IEEE*, Chao Li, Xiao Xu, Lili Guo, *Member, IEEE*, Ling Ding, and Xindong Wu, *Fellow, IEEE*

***Abstract*—Density peaks clustering (DPC) is one of the density-based clustering algorithms and has been widely studied and applied in recent years because of its unique parameter, non-iteration and good robustness. However, it cannot effectively identify the cluster centers, and time and space complexities are too high. To this end, this paper proposes a fast density peaks clustering algorithm based on approximate *k*-nearest neighbors (FDPAN). Firstly, it uses Balanced K-means based Hierarchical K-means (BKHK) method to partition the data and quickly find the approximate *k*-nearest neighbors (AKNN), improving the algorithm's efficiency on large-scale high-dimensional data. Meanwhile, three-way clustering is used to improve the neighbor search of the boundary points of the partition. Then, the local density and relative distance of DPC are recalculated by AKNN. Finally, according to the similar density chain, the connected high-density points are labeled while searching for the cluster center, and the remaining points are assigned to the clusters where their nearest higher-density points are located. Theoretical analysis and experiments on synthetic and real datasets show that FDPAN can obtain higher clustering results and shorten the operation time on large-scale high-dimensional data compared with DPC and its variants.**

***Index Terms*—Density peaks clustering, approximate *k*-nearest neighbors, three-way clustering, similar density chain, efficient clustering.**

I. Introduction

CLUSTERING is an unsupervised learning method and is used to group a set of data points such that points within the same cluster have higher similarity and points between different clusters have lower similarity. It has been applied in many fields, such as market segmentation [1], image recognition [2], gene expression data analysis [3], etc.

The density-based clustering identifies the cluster structure by the densities of points. The core idea is that points in high-density regions are grouped into clusters, while low-density regions are treated as boundaries or noise between clusters. This clustering is suitable for discovering clusters of arbitrary shapes and can deal with noise effectively. Density-based spatial clustering of applications with noise (DBSCAN) [4] is a classical density-based clustering and has strong processing ability for non-linear data. It does not require the number of clusters and can find outliers. However, DBSCAN requires joint adjustment of the neighborhood radius *eps* and the minimum number of points within the radius *minpts*, significantly reducing its usability. Moreover, it has strong parameter sensitivity, and the high time and space complexities also make it difficult to handle large datasets.

Density peaks clustering (DPC) [5] is another representative density-based clustering. It calculates local density ρ_i and relative distance δ_i that play a key role in selecting cluster centers and the assignment of non-centers. The parameter of DPC is unique and only the truncation distance d_c needs to be set. However, DPC, like DBSCAN, has some disadvantages that are common in density-based clustering. 1) DPC is inefficient. Its time and space complexities are $O(n^2)$, limiting the practicality of DPC on datasets with large capacity. 2) The cluster centers selected by DPC cannot avoid the impact of data with uneven density and are often located in high-density clusters in the data.

Researchers have proposed improvements to DPC. In terms of efficiency, the combination of DPC and *k*-nearest neighbors (KNN) is one of the current research hotspots. Cheng et al. [6] improved the running speed of DPC through the fast neighbors search of kd-tree and proposed clustering with local density peaks-based minimum spanning tree (LDP-MST). LDP-MST takes the highest density point in the KNN of the data as the representative and generates sporadic clusters. The clusters are partitioned into multiple disconnected components by MST. Clustering by pruning a density-boosting cluster tree of density mounts (DEMOS) [7] uses KNN to divide data while also considers high-dimensional datasets, because kd-tree becomes less efficient as the number of dimensions increases. However, the matrix operations used by DEMOS to process high-dimensional data take up more storage space, and cannot run once the data exceeds the memory requirements. Li et al. [8] proposed fast DPC based on improved mutual KNN and sub-cluster merging (KS-FDPC). However, KS-FDPC needs two times of KNN, so the overhead of time and space will increase. It does not solve the problem of low efficiency on high-dimensional data. The KNN method has been used in other methods besides DPC.

Received 13 January 2025; revised 17 June 2025; accepted 13 July 2025. Date of publication 16 July 2025; date of current version 15 September 2025. This work was supported by the National Natural Science Foundation of China under Grant 62276265, Grant 62206296, and Grant 62406326. Recommended for acceptance by Farnoush Banaei-Kashani. *(Corresponding author: Chao Li.)*

Shifei Ding, Chao Li, Xiao Xu, and Lili Guo are with the School of Computer Science and Technology/School of Artificial Intelligence, China University of Mining and Technology, Xuzhou 221116, China, and also with the Mine Digitization Engineering Research Center of Ministry of Education, Xuzhou 221116, China (e-mail: dingsf@cumt.edu.cn; chaoli@cumt.edu.cn; xu_xiao@cumt.edu.cn; liliguo@cumt.edu.cn).

Ling Ding is with the College of Intelligence and Computing, Tianjin University, Tianjin 300350, China (e-mail: dltjdx2022@tju.edu.cn).

Xindong Wu is with the Key Laboratory of Knowledge Engineering with Big Data (the Ministry of Education of China), Hefei University of Technology, Hefei 230009, China (e-mail: xwu@hfut.edu.cn).

Digital Object Identifier 10.1109/TKDE.2025.3589794

1041-4347 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.

Sarfraz et al. [9] used the nearest neighbor in the hierarchical clustering to avoid setting the parameters of the algorithm. Cheng et al. [10] used KNN to partition the data and then used each partition block as an input object to speed up the spectral clustering. In addition to accurate KNN search, fast clustering algorithms based on approximate KNN [11] and data partition [12] have also appeared. Approximate KNN sacrifices a small amount of accuracy for computational speed while clustering based on data partition greatly reduces the amount of data by generating coarse-grained subsets. But how to ensure that the algorithm is fast while its effectiveness remains unchanged or even further improved is still underexplored.

There are many variants on how to accurately select the cluster centers of DPC. Ding et al. [13] eliminated the density differences between different clusters by the variance of data, increasing the probability that points in low-density clusters are selected as cluster centers. Liu et al. [14] used Shared Nearest Neighbors to increase the relative distances of low-density points, highlighting these points in the decision graph of DPC. Guan et al. [15] divided the data into several small clusters by KNN, and classified the local density peaks in these clusters as main or satellite peaks. This method mainly highlights the main peaks in decision graph to increase the probability of selecting them. Ding et al. [16] proposed an adaptive cluster merging method that automatically completes clustering without relying on decision graph or manual intervention. Abbas et al. [17] formed multiple connected regions by deleting boundary points. The cluster structure in each connected region is well maintained. Although some effective center selection methods are proposed, many of them rely on kd-tree, resulting in low efficiency on large-scale high-dimensional data.

This paper proposes a fast density peaks clustering algorithm based on approximate KNN (FDPAN) to address the low efficiency of kd-tree on large-scale high-dimensional data. It adopts the Balanced K-means based Hierarchical K-means (BKHK) method [18] to divide the data into different subsets, and the points in each subset are regarded as the approximate neighbors to each other. FDPAN also introduces the idea of three-way clustering (3WC) to BKHK to ensure that the approximate KNN is closer to the real situation. In addition, FDPAN connects the selected center with adjacent high-density points to form core set and increase the probability that the low-density points are selected as centers. The main contributions are as follows.

1) Use BKHK to divide the dataset and select the other points in the final divided subset as approximate KNN. Meanwhile, 3WC is introduced into BKHK to increase the accuracy of approximate KNN of subset boundary points.
2) The approximate KNN is used to accelerate DPC and replace the kd-tree method which is inefficient on large-scale high-dimensional data.
3) Identify the core sets of existing centers by the similar density chain [19], and increase the probability of the low-density point being selected as the cluster center.
4) Experiments are carried out on synthetic and real datasets to verify the improvement of the new method in selecting cluster centers and its effectiveness on large-scale high-dimensional data, respectively.

TABLE I
NOTATIONS AND DEFINITIONS

Symbol	Definition
$X \in \mathbb{R}^{n\times m}$	The dataset with n points.
ρ_i	The local density of x_i.
δ_i	The relative distance of x_i.
γ_i	The decision value of x_i.
g	Indicator vector in BKHK.
$e_i, i=\{1,2\}$	The squared Euclidean distances between all points to be divided and the i-th center.
C_k^+	Positive region in three-way clustering.
C_k^B	Boundary region in three-way clustering.
l	The depth of the binary tree.
D_{ol}	The overlap degree.
$aknn(i)$	The approximate KNN of x_i.
$Rep(i)$	The representative of x_i.
IC	The initial sub-cluster centers.
SDC	Similar density chain.

The rest is structured as follows: Section II reviews related algorithms such as DPC, BKHK and 3WC. The new algorithm is described in Section III. Section IV shows the clustering results on synthetic and real datasets and provides analysis. Finally, the paper and future work are summarized.

II. Related Works

A. Notations

Table I lists the main symbols used in the following parts.

B. Density Peaks Clustering

DPC [5] divides the data into centers and non-central points by local density and relative distance. Cluster centers are preferentially assigned labels, and then labels are assigned to the rest points based on the closest higher-density point. Local density ρ_i and relative distance δ_i are two important variables. When the given dataset is $X = \{x_1, x_2, \ldots, x_n\} \in \mathbb{R}^{n\times m}$, the two variables are given in (1) and (2), respectively.

$$\rho_i = \begin{cases} \sum_{j\neq i} \chi\left(d\left(i,j\right) - d_c\right), & \text{large-scale} \\ \sum_{j=1,\, j\neq i}^{n} \exp\left(-\left(\frac{d(i,\,j)}{d_c}\right)^2\right), & \text{small-scale} \end{cases} \tag{1}$$

$$\delta_i = \begin{cases} \min\limits_{\exists j:\ \rho_j > \rho_i} d\left(i,j\right) \\ \max\limits_{\forall j:\ \rho_j < \rho_i} d\left(i,j\right) \end{cases} \tag{2}$$

where $d(i, j)$ is the Euclidean distance between the i-th and j-th data. $\chi(\cdot)$ is the indicator function. For $x<0$, $\chi(x) = 1$; Otherwise, it is 0. d_c is the truncation-distance that controls the neighbors that play a key role in the density calculation.

The decision graph of δ with respect to ρ is then drawn to facilitate the artificial selection of cluster centers. DPC also calculates the decision value $\gamma_i = \rho_i \cdot \delta_i$ for each data and automatically selects the cluster centers by some maximal γ_i.

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.

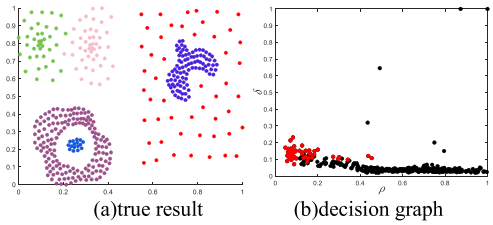


Fig. 1. Decision graph of DPC on Compound.

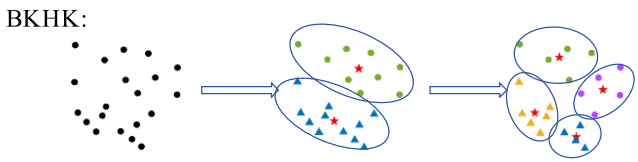


Fig. 2. Example of BKHK.

DPC requires the distance between any point pairs so that the time and space complexities are $O(n^2)$, Greatly reducing the practicability of DPC on large-scale data. In response, fast DPC algorithms based on different strategies have been proposed. These include methods based on grid partition [20], [21], sampling-based methods [22], KNN-based methods [6], [11], [14], [15], and methods based on parallel computing [23]. In addition, there exist some other fast DPC algorithms. For example, Xu et al. [24] introduced the triangle inequality into DPC to avoid the distance calculation between non-neighbor points, and Lu et al. [25] changed the encoding method of data and mapped multi-dimensional data to one-dimensional space to accelerate the neighbors' search. Among the above algorithms, KNN-based ones occupy a considerable part, and kd-tree is a fast KNN search method that is widely used. Therefore, kd-tree plays an important role in the improvement of DPC. However, its efficiency decreases with the increase of dimension. For example, Qiu et al. [11] showed that when the dimension exceeds 10, the time of kd-tree increases rapidly. We aim to design an approximate KNN to compensate for the deficiency.

When selecting centers, DPC prefers high-density clusters and ignores low-density clusters. Fig. 1 shows the dataset with varying densities and the decision graph of DPC on this data. The red points in Fig. 1(b) correspond to the red cluster in Fig. 1(a). As shown in Fig. 1, points in the red low-density cluster have smaller local densities and relative distances, thus bringing difficulties in accurately identifying the cluster centers. Although there are some improved algorithms [13], [14], [15], [16], they need to calculate KNN by kd-tree, so their efficiency is insufficient on complex data.

C. Balanced K-means Based Hierarchical K-means

BKHK [18] is a fast method for partitioning data by combining balanced binary tree and bisecting k-means. It iteratively partitions the data into two subsets, increasing the similarity within each subset and balancing the size between subsets (with equal or nearly equal numbers of points), as shown in Fig. 2. Given the partition depth l, 2^l subsets will be generated. At each partition, the simplified objective function is shown in (3).

$$\min_{g\in\{0,1\},\ \mathbf{1}^{\mathrm{T}}g=\kappa} g^{\mathrm{T}}(e_1-e_2), \tag{3}$$

where column vectors e_1 and e_2 are the squared Euclidean distances between all points to be divided and the two centers, respectively. $\mathbf{1}$ is a column vector where all elements are 1. $\kappa = \lfloor n'/2 \rfloor$ and n' denotes the number of points to be divided. g is (column) indicator vector from all points to the first cluster. If $g_i = 1$, the i-th point is assigned to the first subset; Otherwise, it is assigned to the second subset. After the new subset is formed, the mean of all points in the subset is used as the new center. Update the centers repeatedly and stop when the number of iterations is reached or convergence is achieved. To minimize (3), $g_i = 1$ when the i-th element of $e_1 - e_2$ is the $\lfloor n'/2 \rfloor$ minimum of all its elements.

BKHK is proposed to reduce the time complexity of k-means under massive clusters, so it has a faster partition speed [26]. BKHK is often used to generate high-quality anchors and reduce the amount of data that needs to be processed, thus speeding up the clustering process [27], [28]. However, we find that the points within each generated subset have high similarity and can be used as approximate neighbors of each other, greatly reducing the calculation of the distance between non-nearest neighbor points. The details are described in Section III.A.

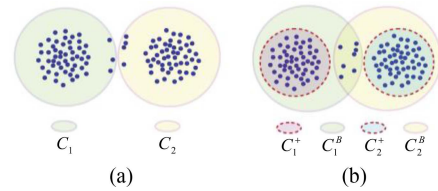


Fig. 3. Comparison of two-way clustering (a) and three-way clustering (b) [33].

D. Three-Way Clustering

3WC [29] combines three-way decision with traditional clustering to deal with uncertainty. Its key idea is to divide the data into three classes: positive regions that clearly belong to a cluster, negative regions that clearly don't belong to a cluster, and boundary regions that may belong to a cluster. The positive and boundary regions of the k-th cluster are denoted by $C_k{}^+$ and $C_k{}^B$ respectively, and the k-th cluster is denoted by $C_k = (C_k{}^+, C_k{}^B)$ in 3WC. Compared with two-way clustering using $C_k = C_k{}^+$ as the k-th cluster, this method not only retains the core advantages of traditional clustering but also can better deal with noise and fuzzy areas. As shown in Fig. 3, 3WC takes the boundary region between two clusters as a common part, which reduces the possibility of mis-assignment of boundary points.

Based on the advantages of 3WC in dealing with boundary points and noise, many improved DPCs also introduce it. Yu et al. [30] expanded the core area in the neighbors of the cluster center, increasing the stability of the center in representing the cluster structure. Wang et al. [31] reduced the dependence of DPC on the number of neighbors by the stability of 3WC and realized adaptive DPC. Ju et al. [32] dealt with

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.

the "Domino" effect in DPC by 3WC, that is, they adopted a two-stage non-central point allocation to avoid the influence of allocated points on subsequent points. Sun et al. [34] adopted 3WC to replace two-way clustering for boundary point division, increasing the probability of correct classification of boundary points. Zhang et al. [35] solved the shortcomings of Euclidean distance in high-dimensional data by isolation kernel, extended positive regions of the centers and determined boundary regions in KNN by the isolation kernel similarity. Ding et al. [36] used 3WC to address the deficiency of DPC in overlapping cluster boundaries for data classification and simplification. The new method is used in online reviews of new energy vehicles to improve the purchase experience and satisfaction. Meng et al. [37] improved DPC's outlier identification accuracy by introducing sinusoidal fuzzy entropy and genetic algorithm to optimize the threshold, and divided each cluster using 3WC to detect outliers more accurately. Although the above methods involve many aspects of DPC, such as parameter optimization, cluster center selection, and assignment of non-center points, they mainly use 3WC to improve the effect of DPC. However, 3WC's application is not limited to this. The new method uses 3WC to improve the accuracy of approximate KNN search, improving the efficiency and effect of the algorithm.

III. FDPAN Algorithm

FDPAN is introduced in detail in this section. Firstly, BKHK based on 3WC is used to quickly search the approximate KNN. Two important variables of DPC are then re-improved by approximate KNN. Finally, the new center selection method is used to find the centers in low-density clusters.

A. BKHK Based on Three-Way Clustering (3WBKHK)

Considering the importance of KNN in DPC, we design the 3WBKHK to improve the efficiency of DPC on large-scale high-dimensional data and the accuracy of approximate KNN. It was mentioned earlier that the points within each subset generated by BKHK can be regarded as the approximate neighbors of each other, as shown in Fig. 2. For the four subsets generated with depth $l = 2$, the points within each subset have high similarity, so each point can be regarded as an approximate neighbor of others in the same subset. The closer the point is to the center of the subset (the red star), the closer its approximate neighbors are to its actual neighbors.

The following problems arise when using BKHK directly for approximate KNN. First, there is the problem of selecting the number of nearest neighbors. If BKHK could equally divide all points into all subsets, it would be easy to identify the shared number of neighbors k. However, this is often not possible due to the complexity of data distribution. In the four subsets (as shown in Fig. 2), the number of points in the orange subset is not equal to that in the blue subset. Second, the accuracy of approximate KNN of the boundary points is not high. Still taking the four subsets as an example, the actual neighbors of boundary points in the purple subset will be more located in blue or green subsets.

3WBKHK is proposed to compute approximate KNN more accurately. When 3WBKHK partitions data, a small number of boundary points in a subset are repeatedly divided into adjacent subsets to find approximate KNN across subsets, which is consistent with 3WC in Fig. 3(b) and makes boundary points and their real neighbors in the same subset. The objective function of 3WBKHK is shown in (4).

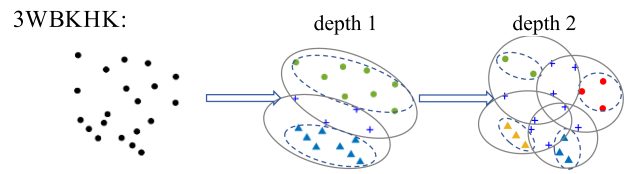


Fig. 4. Example of 3WBKHK (D_{ol} =2).

$$\min_{g_1,\, g_2 \in \{0,1\},\, \mathbf{1}^{\mathrm{T}} g_1 = \mathbf{1}^{\mathrm{T}} g_2 = \alpha} g_1^{\mathrm{T}}(e_1 - e_2) + g_2^{\mathrm{T}}(e_2 - e_1), \quad (4)$$

where g_1 and g_2 are the (column) indicator vectors from all points to two centers, respectively. Unlike BKHK, g_1 only controls whether a point is assigned to the first cluster, that is, when $g_{1,i}$(the i-th element of g_1) $= 1$, the i-th point is assigned to the first cluster; Otherwise, nothing is done. Similarly, the points in the second cluster can be obtained by g_2. $\alpha = \lfloor \frac{n'+1}{2} \rfloor + D_{ol}$. D_{ol} is the overlap degree that indicates how many points within a subset are repeatedly present in its adjacent subsets. n' is the number of points to be divided. The other symbols in (4) are the same as in (3). See part *S4* of supplementary material for the analysis of the convergence of 3WBKHK.

When partitioning data, 3WBKHK obtains the total similarity ($e_1 - e_2$ in (4)). The α points with the highest total similarity and the α points with the lowest total similarity are divided into adjacent subsets, respectively. Therefore, the two subsets have the same cardinality, making it easier to determine the shared number of neighbors. Fig. 4 is an example of 3WBKHK, where '+' is the boundary points that repeatedly exist in adjacent subsets, the dashed line indicates $C_k{}^+$ of 3WC, and the solid line is the union of $C_k{}^+$ and $C_k{}^B$. When the depth of the binary tree is 2, 3WBKHK obtains four subsets with same capacity 8, thus the shared number of neighbors $k = 7$. The sharing of boundary points improves the accuracy of the approximate KNN.

The depth l of 3WBKHK is closely related to the number of nearest neighbors k. However, k involves the improvement of DPC, so we discuss l in detail when analyzing the complexity in Section III.D. $D_{ol} = \lceil n/2^l \rceil$ is used to automatically handle different datasets. Part *S1* of supplementary material indicates that $D_{ol} = \lceil n/2^l \rceil$ or $\lceil n/2^{l+1} \rceil$ and ablation experiments in Section IV.D show that $D_{ol} = \lceil n/2^l \rceil$ achieves better results.

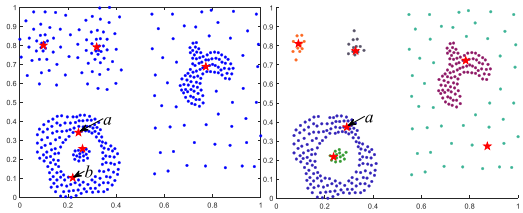
B. Improved Local Density and Relative Distance

To improve the efficiency of DPC, we obtain the approximate KNN and their Euclidean distances through 3WBKHK, and design new ρ_i as shown in (5).

$$\rho_i = \begin{cases} \frac{k}{\sum_{j \in aknn(i)} d(i,j)^p}, & m \leq 100 \\ |\{x_j | i \in aknn(j)\}|, & \text{otherwise} \end{cases} \quad (5)$$

where $aknn(i)$ represents the approximate KNN of the i-th point and $d(i, j)$ is the Euclidean distance between two points.

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 5. Center points selected by DPC (a) and SDC (b) on compound dataset.

m is the dimension. Because of the curse of dimensionality [38], Euclidean distance will fail in high-dimensional space, so FDPAN uses different ρ_i in (5). The utility of reverse nearest neighbors is analyzed in high-dimensional space [39], [40]. In FDPAN, the local density calculated by the number of reverse k-nearest neighbors is beneficial to the subsequent clustering, especially for building similar density chains. To further improve the stability of FDPAN under different k when the dimension is low, the density adjustment strategy in DEMOS [7] is adopted, that is, keep increasing p such that the coefficient of variation of the local density approaches 0.4. In statistics, the coefficient of variation is a measure of the relative degree of fluctuations and equals the standard deviation divided by the mean.

After obtaining ρ_i by (5), FDPAN partitions the data into different sub-clusters using representatives denoted as $Rep(i) = \text{argmin}_\tau d(i, \tau)$[41], where τ belongs to mutual approximate kv-nearest neighbors of the i-th point and $\rho_\tau > \rho_\text{i}$. The distance of a point to its representative is taken as the new relative distance δ_i. $kv(\leq k)$ is a positive integer used to calculate representative and build similar density chain.

FDPAN selects the points satisfying $Rep(i) = i$ as the initial sub-cluster centers $IC = \{ic_1, ic_2, \ldots, ic_s\}$. Then, the distance matrix of IC is calculated and used as the input of (2) to obtain the relative distance of these centers. As the amount of data increases and kv decreases, s will increase. Therefore, when s exceeds a threshold (i.e., 20000), FDPAN repeats the above processes with IC as initial data, including computing approximate KNN by 3WBKHK, computing representatives, and generating new IC. The smaller the amount of data, the smaller the depth l used to compute the approximate KNN. Since the capacity of IC is greatly reduced compared to the raw data, l of 3WBKHK is continuously reduced to calculate enough nearest neighbors of IC. By using a method similar to hierarchical clustering, the number of centers used to calculate the distance matrix is controlled in a relatively reasonable range, reducing the complexity and speeding up the running process.

C. Cluster Centers Selection

To address the limitations of DPC in selecting cluster centers, we propose a more effective method. This method is based on the fact that after a center is selected, the probability of its approximate KNN with high similarity being selected as centers will be greatly reduced. As shown in Fig. 5(a), DPC identifies six cluster centers (red stars) distributed across five ground-truth clusters. Although centers a and b belong to the same cluster and the points in the cluster have high similarity, the manifold structure of the cluster makes DPC select two high-density points in it as the cluster centers.

To increase DPC's accuracy in selecting cluster centers, we use a similar density chain [19] to expand the selected centers, avoiding the points in the connected high-density region being selected as cluster centers again. The definition of similar density chain is as follows.

Definition 1: Similar density chain (SDC). SDC is a point set, and $SDC_i = \{x_i\}$ for an unlabeled point x_i. Take out the first point x_{top} of SDC_i. If there is an unlabeled point x_j in the kv-nearest neighbors of x_{top} and x_j satisfies $\theta \cdot \rho_\text{i} \leq \rho_\text{j} \leq (1+\theta) \cdot \rho_\text{i}$, add x_j to SDC_i and change the status of x_j to labeled. The first point of SDC_i is taken out again and the above process is repeated until SDC_i is empty. The points labeled by the above process are the similar density chain of x_i.

SDC is used to traverse the region adjacent to the center, where $\theta = 0.4$ ensures that points in SDC have high density similarity. The unlabeled point x_i is traversed according to the descending order of decision values γ_i until the number of SDCs reaches the given number of clusters. Points that are not in any SDC are assigned to the cluster where the nearest higher-density point is located. Fig. 5(b) shows the effect of SDC in selecting cluster centers. After center a is selected, SDC will label other points in the blue ring cluster before selecting the next center, thus reducing the probability of points in the same cluster being selected as centers at the same time. In SDC, the extended core sets are segmented by low-density points, therefore, the local density in (5) is required to have a large standard deviation. On high-dimensional data, the distribution of Euclidean distance is more concentrated, so that the standard deviation of local densities is small. The number of reverse k-nearest neighbors can make up for this deficiency well, and the specific comparison can be seen in part *S5* of the supplementary material.

SDC and density accessibility in DBSCAN are used to connect regions with similar densities. But there are differences. Firstly, SDC starts from the cluster center and searches for points with high-density similarity in the mutual kv-nearest neighbors. However, density accessibility starts from any point and searches for core and boundary points in *eps* radius. Unlike DBSCAN, which needs two parameters, SDC only depends on kv. Secondly, SDC will exclude the points with too high- or too low-density during search process to ensure the high-similarity of the cluster core set. However, density accessibility will group all points with higher density in *eps* radius into one cluster, which is not conducive to processing datasets with variable densities.

D. Algorithm Flow and Analysis

In this subsection, the flow of FDPAN is given and the complexity is analyzed.

In Algorithm 1, 3WBKHK needs depth l, and this parameter can affect k as well as ρ_i in (5). A smaller l leads to a larger k, which results in points in different regions being used to compute ρ_i. For this, FDPAN specifies the minimum of l according to the amount of data n. The analysis is as follows.

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: FDPAN.

Input: Data $X = \{x_1, x_2, \ldots, x_n\}$.
Parameter: The depth *l*, the number of neighbors *kv*, and clusters' number *nclust*.
Output: Clustering result *C*.

1. Divide *X* by 3WBKHK to obtain the approximate KNN;
2. Calculate ρ_i by (5), and calculate δ_i in the mutual *kv*-nearest neighbors by (2) to obtain *Rep*(*i*);
3. Select points with *Rep*(*i*) = *i* as $IC = \{ic_1, \ldots, ic_s\}$, and their distance matrix is used to calculate δ_i. If *s*>threshold, repeat steps 1-3 with *IC* as initial points;
4. Calculate γ_i, sort points in descending order, and obtain the descending index *ds*(*i*), $i = 1, \ldots, n$;
5. **for** $i = 1{:}n$
6. If $x_{ds(i)}$ is not assigned, select it as the new center; otherwise, **continue**;
7. By Definition 1, calculate the $SDC_{ds(i)}$ of $x_{ds(i)}$, points in $SDC_{ds(i)}$ are assigned to the new cluster;
8. If the clusters' number is *nclust*, **break**;
9. **end**
10. Assign the remaining point to the cluster with the nearest higher-density point;
11. Output the clustering result *C*.

Let $k \leq 99$ to minimum *l*, while assume that *n* is a multiple of 2^l for calculation convenience. If the overlap degree $D_{ol} = 0$ in 3WBKHK, the number of points in the subset in depth *l* is $n/2^l = k+1$. Because $D_{ol} = \lceil n/2^l \rceil$, the total number of points needed to be partitioned by 3WBKHK is approximately $n + n/2^l(2 + 2^2 + \ldots + 2^l)$, that is, $n + n/2^l 2(2^l - 1) \approx 3n$. Thus $3n/2^l\text{-}1 = k \leq 99$, $l \geq \log_2(\frac{3n}{100})$. Since *l* is an integer, the minimum of *l* is $\lfloor \log_2(\frac{3n}{100}) \rfloor$.

In addition to setting the depth *l*, the complexity of FDPAN is analyzed. To calculate the time complexity, each step of Algorithm 1 needs to be analyzed. In step 1, the complexity of BKHK is $O(n \cdot m \cdot l \cdot t)$, where *n* and *m* denote the capacity and dimension respectively, and *t* is the number of iterations. Similarly, the complexity of 3WBKHK is $O(3n \cdot m \cdot l \cdot t)$, $t << n$. In addition to partitioning the data, 3WBKHK computes the distance matrices within all subsets with a time complexity of $O(2^l(3n/2^l)^2) = O(9n^2/2^l)$. In step 2, the time complexities of computing ρ_i and δ_i are $O(n \cdot k)$ and $O(n \cdot kv)$, respectively. The distance matrix of the initial sub-cluster centers in step 3 takes $O(s^2)$. The time complexities of step 4 and the remaining for loop are $O(n \cdot log(n))$ and $O(n)$, respectively. In MATLAB, there are very efficient distance matrix calculation methods, so when *s* is less than a threshold, $O(s^2)$ can be ignored. After the above analysis, the total time complexity of FDPAN is about $O(n(3m \cdot l \cdot t + 9n/2^l + k + kv + log(n) + 1))$, $l, t << n, n/2^l \leq 100/3, kv \leq k \leq 100$. The time complexity of FDPAN is about $O(n(3m + \log(n)))$ on large-scale high-dimensional data, so FDPAN is an efficient clustering algorithm.

The space complexity mainly includes the approximate KNN and the *k*-nearest neighbors distance $O(2n \cdot k)$, and the distance matrix $O(s^2)$ of the initial sub-cluster centers *IC*. Therefore, the total space complexity is $O(2n \cdot k + s^2), k \leq 100$, which is at a relatively low level and *s* is controllable.

IV. Experiment and Analysis

In this section, we use experiments to verify FDPAN. The synthetic datasets are used to test the superiority of the centers selected by FDPAN, and the real datasets are used to verify the effect and efficiency. FDPAN requires max-min normalization for preprocessing.

The accuracy (ACC) [42], adjusted rand index (ARI) [42], adjusted mutual information (AMI) [16] and running time (RT) are used as evaluation indicators. ACC, ARI and AMI are used to measure the effect, and RT is used to measure the efficiency. ACC is the ratio of the number of correctly clustered points to the total number of points. The larger the ratio, the more accurate the clustering results, and the maximum is 1. ARI represents the clustering result by the number of correct partitions (e.g., whether the points of the same class are classified into the same cluster or whether the points of different classes are classified into different clusters). AMI uses the information entropy of the actual class labels and the clustering results to compare the good and bad clustering results. ARI and AMI lie in [-1, 1], where higher values reflect better consistency with the ground truth. RT is taken as the average of ten run times in seconds.

The environment of all experiments is: Windows 11 operating system, MATLAB R2021b compiler, i9-12900HX processor and 16 GB RAM.

A. Results and Analysis on Synthetic Data

The centers selected by DPC [5] and FDPAN are compared through synthetic data. Figs. 6 and 7 show the selected cluster centers (red stars) and clustering results of DPC and FDPAN, respectively, where different clusters are represented by different colors. The parameter interval of DPC is set as [0.1, 6]% and the step size is 0.1% to increase the adaptability on complex data. The depth *l* of FDPAN is continuously increased from the minimum until the number of neighbors *k*<2. *kv* is an integer in [1, *k*] and the number of clusters is set to the true value.

The data in Fig. 6 primarily consist of datasets with density variations. Among these, the Compound, Jain, and Eyes show significant differences in local densities. In addition, the data contain different numbers of manifold clusters, and the complex data structure can easily lead DPC to over-partition points within the same class. We compare DPC and FDPAN on these data to fully illustrate the effectiveness of FDPAN.

The Compound data has five high-density clusters and one low-density cluster distributed around the high-density cluster, which makes it difficult to effectively identify the points in the low-density cluster. DPC selects the cluster centers by 6 maximum decision values, but only identifies five high-density clusters. The low-density cluster is mistakenly assigned to the cluster with high-density points in its middle. FDPAN accurately selects the points in six clusters as centers. This is because after

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.

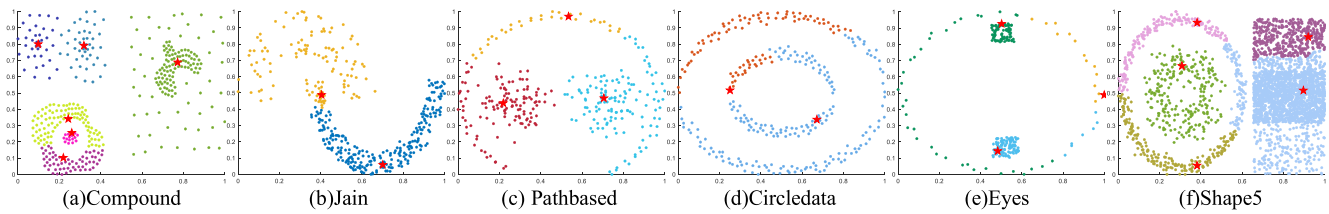


Fig. 6. Cluster centers selected by DPC on synthetic data and clustering results.

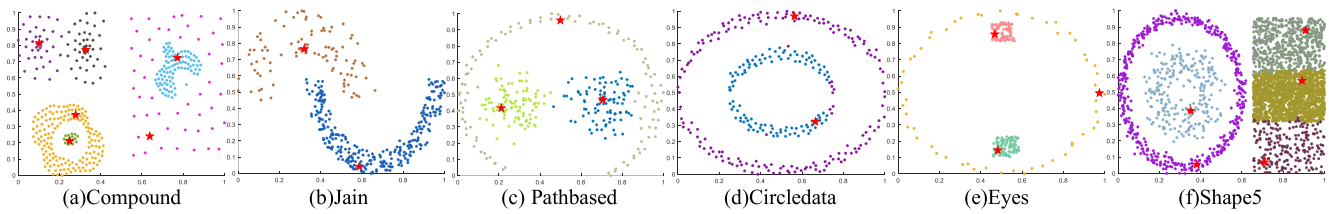


Fig. 7. Cluster centers selected by FDPAN on synthetic data and clustering results.

selecting a cluster center, FDPAN labels the points in the core set of the center through SDC, reducing the probability of points in the same cluster being selected as the cluster centers.

The Jain dataset contains two arc-shaped clusters, in which the intra-cluster density is relatively uniform and the inter-cluster density is significantly different. DPC selects two centers in the high-density cluster, causing over-segmentation of the cluster and incorrect allocation of low-density cluster. Because FDPAN adopts a more excellent center selection mechanism, it accurately identifies the centers of two clusters and achieves correct results.

Pathbased has a circular low-density cluster outside two convex clusters. Both DPC and FDPAN can identify the centers of different clusters on this data. However, when DPC processes the points far from the center in the circular cluster, it mistakenly clusters these points into the boundary region of adjacent high-density clusters. FDPAN enhances results by first identifying the circular cluster's center, then pre-labeling distant points on the circular cluster, and finally assigning boundary points of the high-density clusters.

In addition to clusters with different densities, Circledata also has low-density regions within the cluster. On the data, DPC selects two points of the high-density inner ring cluster as centers, resulting in both inner and outer ring clusters being divided into two sets. In contrast, FDPAN identifies centers in two clusters and achieves nearly perfect clustering results. The disadvantage of FDPAN is that sparse boundary points are incorrectly assigned from the inner cluster to the outer cluster, leading to inaccuracies compared to the ground truth.

Eyes and Pathbased have similar data structures, but exhibit greater inter-cluster differences. Although DPC and FDPAN identify correct cluster centers on Eyes, FDPAN achieves optimal results on low-density ring cluster due to the larger inter-cluster density difference.

On Shape5, points from different clusters have a larger overlap, making it difficult to identify adjacent clusters with different densities. DPC selects two points in the ring cluster as centers, merging points from different density regions. In contrast, FDPAN's centers align with the ground truth, accurately separating clusters in different density regions.

Experiments on synthetic data show that although FDPAN has some errors in boundary points, it significantly reduces the bias of DPC in selecting cluster centers and obtains results that are very consistent with the real situation. Therefore, FDPAN is effective in improving center selection and enhancing overall clustering performance.

TABLE II
REAL DATASETS

Datasets	Capacity	Dimension	Class
wine	178	13	3
sonar	208	60	2
seeds	210	7	3
pima	768	8	2
segmentation	2310	19	7
abalone	4177	8	3
ringnorm	7400	20	2
pendigits	10992	16	10
Gamma	19020	10	2
avila	20867	10	12
madelon	2000	500	2
Gisette	7000	5000	2
CIFAR100	60000	3072	100
FMNIST	70000	784	10
ijcnn	141691	22	2
Covertype	581012	54	7

B. Results and Analysis on Real Data

We also examine FDPAN on real data. The details of these real datasets are shown in Table II, where madelon, Gisette, CIFAR100, FMNIST, ijcnn, and Covertype are datasets with

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.

higher dimensions or larger capacities. FMNIST stands for image data Fashion-MNIST.

In addition to DPC [5], we adopt some modified DPC algorithms, including DCDP-ASC [10], MDPC+ [43], KS-FDPC [8], SMMP [44], DBSCAN-DPC [45], NDP-Kmeans [46], and Fast LDP-MST [11]. Except for DBSCAN-DPC that calculates the distance matrix, others use the kd-tree to accelerate the running process. MDPC+, SMMP, and Fast LDP-MST use the hybrid KNN methods. For low-dimensional data, these three algorithms use efficient kd-tree, whereas when the dimension grows, MDPC+ and SMMP adopt fast matrix operations, and Fast LDP-MST uses RP-forest for fast approximate KNN. Therefore, the selected algorithms can be used to compare the efficiencies of 3WBKHK and kd-tree on large-scale high-dimensional data.

Parameter setting: DCDP-ASC needs to select centers by the decision graph and the ratio coefficient φ. $\varphi = 0$ or 0.1 indicates whether a small amount of noise needs to be removed. In addition to manually selecting centers, MDPC+ uses the number of nearest neighbors k. It provides a default $k = round(\sqrt{n})$. Additionally, the optimal result within [5, 6, …, 30]is selected and compared with that under default conditions, with the superior result retained as output. KS-FDPC needs the number of nearest neighbors k and the number of clusters *nclust*. We set k for KS-FDPC by the original paper [8]. The parameter-free SMMP achieves better results when *nclust* is given. DBSCAN-DPC uses the default parameters from the original paper [45], which is only valid for two-dimensional data. To handle complex data, DBSCAN-DPC recommends t-sne preprocessing, but this increases runtime. NDP-Kmeans replaces the decision graph with the decision value and uses *nclust* to pick out centers. It also requires a ratio coefficient for noise, like in DCDP-ASC. The k of Fast LDP-MST is selected on the default $ceil(\log_2(n))$ or [5, 6, …, 30], and $k = [10, 20, \ldots, 100]$ on complex datasets FMNIST and CIFAR100.

Table III shows the comparison results of all algorithms on real datasets. As shown in Table III, FDPAN achieves the best results on over half of the datasets and has significant improvement compared to the suboptimal algorithms on pima, ringnorm, Gamma, and Gisette. Although FDPAN does not achieve optimality on some datasets, it has good results. On sonar, FDPAN not only attains the highest AMI but also achieves ACC and ARI second only to KS-FDPC and SMMP. On segmentation, while FDPAN's ACC is slightly lower than that of DBSCAN-DPC, its ARI and AMI significantly outperform others. On pendigits, although DBSCAN-DPC has the highest ACC, its ARI and AMI are inferior to those of KS-FDPC. The results of FDPAN are weaker than those of KS-FDPC and DBSCAN-DPC but better than others. On the larger high-dimensional datasets (CIFAR100, FMNIST, ijcnn, and Covertype), only NDP-Kmeans, Fast LDP-MST, and FDPAN output clustering results, while others make errors. On these four datasets, the results of FDPAN on CIFAR100 and ijcnn are optimal. On FMNIST, the AMI of FDPAN is slightly lower than that of KS-FDPC. On Covertype, FDPAN's result is weaker than that of Fast LDP-MST. Although DCDP-ASC has results on four complex datasets except CIFAR100, it is not effective. Moreover, DCDP-ASC needs to increase the upper limit of the maximum k to ensure the selection of natural characteristic value, which reduces the efficiency.

In general, the clustering results of FDPAN are higher than those of others, thus demonstrating its effectiveness.

This section also presents the comparison of running time on large-scale or high-dimensional datasets to verify the efficiency of FDPAN, as shown in Table IV. Time comparison on more datasets is provided in part *S2* of the supplementary material.

As shown in Table IV, FDPAN is more efficient on large-scale high-dimensional data. Among these data, madelon and Gisette have lower capacities, enabling DPC, MDPC+, and SMMP to quickly compute the distance matrix through matrix operations. On CIFAR100, FMNIST, ijcnn, and Covertype, the efficiency of FDPAN reaches the best among all algorithms, except for the time of FDPAN is slightly higher than that of Fast LDP-MST on ijcnn. The matrix operations used by DPC, MDPC+, and SMMP consume excessive memory on the four datasets, and the programs cannot run.

In addition to Fast LDP-MST and FDPAN, NDP-Kmeans can also output clustering results on four large-scale high-dimensional datasets, but the kd-tree is far less efficient than the approximate KNN used by Fast LDP-MST and FDPAN. On CIFAR100 and FMNIST, NDP-Kmeans needs to increase the upper limit of the maximum k when calculating the natural characteristic value to ensure operation. However, this will increase the time of NDP-Kmeans.

DCDP-ASC can process FMNIST, ijcnn, and Covertype, but its time is much higher than those of other algorithms, which is caused by the inefficiency of kd-tree on large-scale high-dimensional data and the time complexity of $O(sn^3)$ in spectral clustering. sn is the number of initial sub-clusters and increases with data volume. Similarly, KS-FDPC can handle the FMNIST dataset, but it is very inefficient compared with Fast LDP-MST and FDPAN.

See *S3* in the supplementary material for more analysis of the resource usage of FDPAN.

C. Statistical Test

In this section, we conduct Friedman's tests by the results in Tables III and IV to illustrate the significant difference of FDPAN. The test defines the null hypothesis (H_0) as "no significant differences exist among all groups" and the alternative hypothesis (H_1) as "at least two groups differ significantly." When Friedman's test yields a p-value less than $\alpha = 0.05$, H_0 is rejected, indicating significant differences. The p-values between algorithms are listed in the upper triangle of Tables V, VI, VII , and VIII, where DPC, DCDP-ASC, MDPC+, KS-FDPC, SMMP, DBSCAN-DPC, NDP-Kmeans, Fast LDP-MST, and FDPAN are abbreviated as A1, A2, A3, A4, A5, A6, A7, A8, A9, respectively.

Tables V, VI, and VII show that only A6(DBSCAN-DPC) has no significant difference with A9(FDPAN) in ACC, which fully proves that FDPAN can achieve better results. DBSCAN-DPC does not need the number of clusters and may generate more clusters, resulting in over-segmentation of the data. This will reduce the ARI and AMI of DBSCAN-DPC and affect its stability while leading to higher ACC. According to Table VIII,

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.

TABLE III
CLUSTERING RESULTS ON REAL DATASETS

Algorithm	ACC	ARI	AMI	ACC	ARI	AMI	ACC	ARI	AMI	ACC	ARI	AMI
	wine			sonar			seeds			pima		
DPC	0.6910	0.3910	0.4247	0.5769	0.0190	0.0161	0.8952	0.7170	0.6715	0.6510	0.0000	0.0000
DCDP-ASC	0.7247	0.4007	0.3884	0.5433	0.0041	0.0207	0.8381	0.5893	0.6150	0.6510	0.0119	0.0013
MDPC+	0.9382	0.8170	0.8054	0.6154	0.0487	0.0429	0.9000	0.7266	0.6701	0.6732	0.0812	0.0348
KS-FDPC	0.9157	0.7562	0.7600	**0.6298**	**0.0629**	0.0521	0.9048	0.7448	0.7284	0.6823	0.0739	0.0451
SMMP	0.8933	0.6990	0.7233	0.6250	0.0580	0.0446	0.8952	0.7158	0.6618	0.6510	0.0323	0.0071
DBSCAN-DPC	0.6517	0.3351	0.4446	0.5288	-0.0035	0.0393	0.8857	0.6201	0.6528	0.6745	0.0115	0.0339
NDP-Kmeans	0.9214	0.7703	0.7720	0.5289	-0.0012	-0.0060	0.9095	0.7533	0.7133	0.6510	0.0089	0.0002
Fast LDP-MST	0.7135	0.3627	0.4267	0.5289	-0.0035	0.0393	0.8905	0.7136	0.7080	0.6589	0.0196	0.0115
FDPAN	**0.9438**	**0.8368**	**0.8233**	0.6154	0.0494	**0.1534**	**0.9333**	**0.8125**	**0.7670**	**0.7109**	**0.1649**	**0.0911**
	segmentation			abalone			ringnorm			pendigits		
DPC	0.6433	0.4571	0.5991	0.5150	0.1349	0.1482	0.5108	0.0003	0.0004	0.7205	0.5655	0.7350
DCDP-ASC	0.6303	0.4544	0.5954	0.5150	0.1364	0.1320	0.5062	0.0000	0.0025	0.5418	0.3620	0.6597
MDPC+	0.7311	0.5831	0.7142	0.5272	0.1512	0.1589	0.5081	0.0002	0.0062	0.6741	0.5644	0.7448
KS-FDPC	0.6909	0.5632	0.6876	0.4293	0.0244	0.0869	0.5076	0.0001	0.0016	0.8857	**0.7838**	**0.8563**
SMMP	0.6827	0.5271	0.6674	0.5310	0.1638	0.1702	0.5049	0.0000	0.0000	0.8417	0.7050	0.8067
DBSCAN-DPC	**0.7792**	0.3294	0.5632	0.5288	0.0277	0.0870	0.5116	0.0004	0.0113	**0.9612**	0.7305	0.8384
NDP-Kmeans	0.4286	0.2359	0.5712	0.5150	0.1364	0.1320	0.5049	0.0000	0.0000	0.5189	0.2927	0.6340
Fast LDP-MST	0.6234	0.4389	0.7175	0.5166	0.0658	0.1033	0.5078	0.0001	0.0056	0.7927	0.6402	0.7915
FDPAN	0.7684	**0.6438**	**0.7382**	**0.5430**	**0.1914**	**0.1960**	**0.9510**	**0.8134**	**0.7428**	0.8634	0.7480	0.8213
	Gamma			avila			madelon			Gisette		
DPC	0.6484	-0.0009	0.0000	0.4397	0.0428	0.0987	0.5370	0.0050	0.0044	0.5146	0.0008	0.0151
DCDP-ASC	0.6484	0.0000	0.0000	0.4707	0.0878	0.1693	0.5495	0.0094	0.0107	0.5001	0.0000	0.0000
MDPC+	0.6892	0.0698	0.0918	0.4818	0.0561	0.1807	0.5440	0.0072	0.0053	0.7081	0.1732	0.1459
KS-FDPC	0.6646	0.0331	0.0288	0.4154	0.0005	0.0133	0.5080	0.0020	0.0071	0.5007	0.0000	0.0012
SMMP	0.6523	-0.0105	0.0015	0.4836	0.0804	0.2214	0.5030	-0.0002	-0.0004	0.5000	0.0000	0.0000
DBSCAN-DPC	0.6484	-0.0203	0.0076	**0.5196**	0.1184	**0.2459**	0.5395	0.0015	0.0076	0.6931	0.1033	0.2127
NDP-Kmeans	0.6638	0.0269	0.0441	0.4182	-0.0230	0.0461	0.5420	0.0069	0.0182	0.5000	0.0000	0.0000
Fast LDP-MST	0.6848	0.0774	0.0570	0.5019	0.1036	0.2213	0.5145	0.0008	0.0091	0.5287	0.0033	0.0474
FDPAN	**0.7648**	**0.2492**	**0.2031**	0.4800	**0.1210**	0.2216	**0.5770**	**0.0233**	**0.0184**	**0.8463**	**0.4796**	**0.3987**
	CIFAR100			FMNIST			ijcnn			Covertype		
DPC	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
DCDP-ASC	N/A	N/A	N/A	0.2010	0.1079	0.2991	**0.9043**	0.0022	**0.0000**	0.5193	-0.0195	0.0621
MDPC+	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
KS-FDPC	N/A	N/A	N/A	0.4694	0.3447	**0.5524**	ERR	ERR	ERR	N/A	N/A	N/A
SMMP	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
DBSCAN-DPC	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
NDP-Kmeans	0.0100	0.0000	0.0000	0.3111	0.0726	0.3225	**0.9043**	0.0000	**0.0000**	0.4887	0.0017	0.0075
Fast LDP-MST	0.0232	0.0016	0.0426	0.3483	0.0791	0.3710	**0.9043**	0.0000	**0.0000**	**0.5949**	**0.0838**	**0.1924**
FDPAN	**0.0634**	**0.0152**	**0.0974**	**0.5736**	**0.3481**	0.5503	**0.9043**	**0.0024**	**0.0000**	0.5464	0.0644	0.1356

Bold indicates the optimal value, "N/A" means out of memory, "ERR" means running error.

TABLE IV
COMPARISON OF RUNNING TIMES ON REAL DATASETS (IN SECONDS)

Datasets	DPC	DCDP -ASC	MDPC+	KS-FDPC	SMMP	DBSCAN -DPC	NDP-Kmeans	Fast LDP -MST	FDPAN
madelon	0.2279	1.7710	**0.1205**	0.1606	0.1384	3.6470	0.3918	0.2030	0.3208
Gisette	19.6665	236.3202	10.6282	20.1491	17.7743	32.3896	40.1080	12.3130	**9.9872**
CIFAR100	-	-	-	-	-	-	2475.8979	155.7868	**112.5481**
FMNIST	-	3847.6224	-	368.5576	-	-	679.2483	70.8865	**48.6788**
ijcnn	-	525.0029	-	-	-	-	47.3589	**5.3485**	6.8885
Covertype	-	4077.4805	-	-	-	-	410.3687	172.5630	**61.5564**

Bold indicates the optimal value and '-' indicates no result.

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.

TABLE V
FRIEDMAN'S TESTS FOR ACC (%)

	A1	A2	A3	A4	A5	A6	A7	A8	A9
A1		31.73	2.09	56.37	20.59	13.17	52.71	56.37	0.05
A2			0.39	8.33	13.17	13.17	20.59	24.82	0.05
A3				24.82	24.82	100	0.39	2.09	0.67
A4					2.09	56.37	56.37	100	2.09
A5						56.37	73.89	100	2.09
A6							56.37	24.82	8.33
A7								13.17	0.05
A8									0.39
A9									

TABLE VI
FRIEDMAN'S TESTS FOR ARI (%)

	A1	A2	A3	A4	A5	A6	A7	A8	A9
A1		100	2.09	56.37	56.37	56.37	100	56.37	0.05
A2			2.09	13.17	20.59	56.37	31.73	56.37	0.05
A3				8.33	24.82	8.33	0.39	8.33	0.05
A4					3.48	24.82	36.57	36.57	2.09
A5						56.37	52.71	56.37	0.39
A6							100	76.30	0.05
A7								56.37	0.05
A8									0.05
A9									

TABLE VII
FRIEDMAN'S TESTS FOR AMI (%)

	A1	A2	A3	A4	A5	A6	A7	A8	A9
A1		76.30	2.09	8.33	24.82	8.33	56.37	0.39	0.05
A2			0.39	24.82	3.48	8.33	52.71	2.09	0.05
A3				56.37	24.82	56.37	8.33	56.37	0.05
A4					2.09	56.37	56.37	56.37	0.39
A5						56.37	52.71	56.37	0.05
A6							100	76.30	0.39
A7								24.82	0.39
A8									0.05
A9									

TABLE VIII
FRIEDMAN'S TESTS FOR RT (%)

	A1	A2	A3	A4	A5	A6	A7	A8	A9
A1		8.33	100	31.73	100	100	4.55	4.55	4.55
A2			8.33	56.37	8.33	8.33	4.55	4.55	4.55
A3				31.73	100	100	4.55	4.55	4.55
A4					31.73	31.73	31.73	4.55	4.55
A5						100	4.55	4.55	4.55
A6							4.55	4.55	4.55
A7								4.55	4.55
A8									31.73
A9									

A9(FDPAN) is not significantly different from A8(Fast LDP-MST), but differs significantly from all other methods, proving the efficiency of FDPAN on large-scale high-dimensional datasets. The efficiency of Fast LDP-MST and FDPAN is determined by the fast approximate KNN search. Overall, FDPAN is a powerful algorithm.

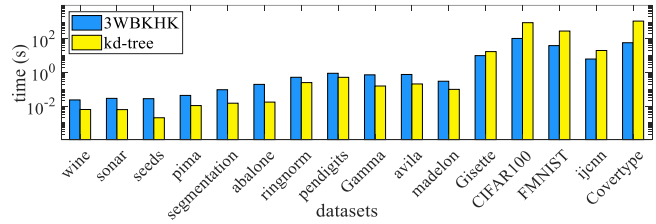


Fig. 8. Comparison between 3WBKHK and kd-tree in time.

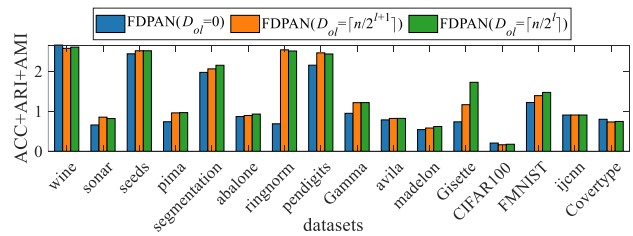


Fig. 9. Clustering results under different overlap degrees.

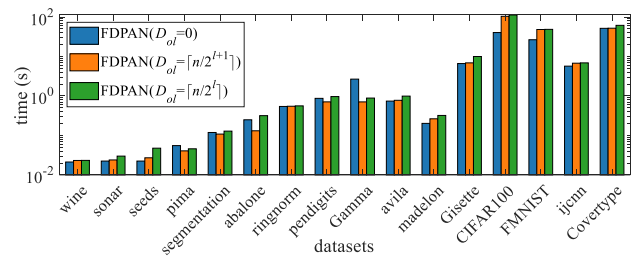


Fig. 10. The running time under different overlap degrees.

D. Ablation Experiment

To verify the efficiency of 3WBKHK and the effectiveness of D_{ol} in 3WBKHK, we conduct ablation experiments.

Fig. 8 shows the comparison of running time between 3WBKHK and kd-tree when searching the same number of nearest neighbors. The search rate of kd-tree is faster than that of 3WBKHK on the first 11 datasets with small dimensions and low capacities. On the remaining five large-scale or high-dimensional datasets, the speed of 3WBKHK is better than that of kd-tree, and the time difference on these five datasets is much higher than that of the previous small-scale datasets. Therefore, 3WBKHK saves more time on large-scale high-dimensional data.

Figs. 9 and 10 show the influence of different D_{ol} on the results and running time of FDPAN, respectively. Fig. 9 shows that $D_{ol} = \lceil n/2^l \rceil$ or $\lceil n/2^{l+1} \rceil$ makes FDPAN achieve better results, especially on ringnorm, pendigits, Gamma, Gisette, and FMNIST. Moreover, compared with $\lceil n/2^{l+1} \rceil$, $D_{ol} = \lceil n/2^l \rceil$ further improves the results on Gisette and FMNIST. Fig. 10 shows that the reduction of D_{ol} will reduce the amount of computation and thus have less running time. While the anomalous cases on pima and Gamma are related to unreasonable parameter settings. Considering Figs. 9 and 10 collectively, $D_{ol} = \lceil n/2^l \rceil$ demonstrates higher rationality, as it significantly enhances clustering accuracy with only a moderate increase in runtime.

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.

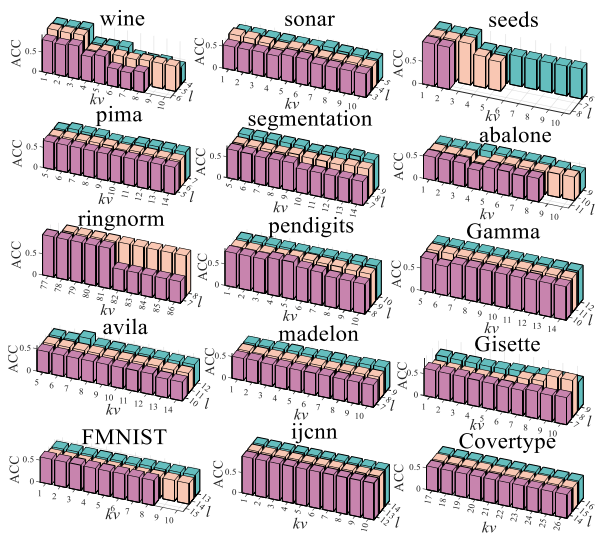


Fig. 11. Effect of parameters l and kv on ACC of FDPAN.

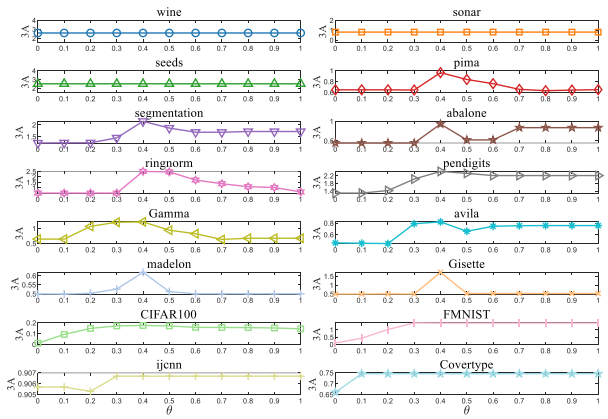


Fig. 12. Effect of parameter θ on clustering results.

E. Parameters Analysis

This section analyzes the parameter sensitivity of FDPAN by examining the effect of parameters l and kv on accuracy. Experiments conducted on 15 datasets from Table II yield results presented in Fig. 11 . The minimum of depth l is $\lfloor \log_2(\frac{3n}{100}) \rfloor$. l is chosen from three consecutive integers centered at FDPAN's optimal value, and the interval of kv is also around the optimal result. Missing bars indicate parameter combinations where FDPAN failed to execute. When $l = 9$, FDPAN has no result for $kv = [77, 86]$ on ringnorm.

Fig. 11 shows that FDPAN has relatively stable results on half of the datasets, such as sonar, pima, pendigits, Gamma, madelon, ijcnn, and Covertype. However, it has strong parameter sensitivity on some datasets due to unreasonable parameter combinations, such as wine, seeds, ringnorm, and Gisette. Nevertheless, FDPAN exhibits relatively stable optimal parameter intervals across most datasets except Gisette and avila, facilitating efficient optimization. On these two datasets, ACC decreases monotonically near optima before converging to plateaus. Overall, FDPAN's parameter sensitivity requires further refinement for robust adaptability.

Fig. 12 shows the effect of different θ on 3A (= ACC+ARI +AMI). Significant θ-sensitivity is observed for datasets including pima, segmentation, abalone, ringnorm, Gamma, madelon, and Gisette. Crucially, $\theta = 0.4$ can make FDPAN have higher clustering results on all datasets. On pima, segmentation, pendigits, madelon, and Gisette, FDPAN has the best performance only on $\theta = 0.4$, which fully illustrates the rationality of $\theta = 0.4$. From the perspective of the expansion of the core set, $\theta = 0.4$ ensures that the neighborhood area of the cluster center is fully contained, and the adjacent boundary will not be introduced. Therefore, $\theta = 0.4$ is used to avoid human interference.

In summary, FDPAN represents a significant improvement over the DPC algorithm.

V. Conclusion

This paper proposes a fast density peaks clustering based on approximate KNN to enhance the efficiency of DPC on large-scale high-dimensional datasets. Firstly, FDPAN integrates three-way clustering with BKHK to refine the accuracy of the approximate KNN. Secondly, it leverages approximate KNN to rapidly compute two important variables of DPC. Finally, core sets of the cluster centers are determined via similar density chain, reducing the probability of the points in the same cluster being selected as the cluster centers at the same time. Experimental results comprehensively validate FDPAN's cluster center accuracy, effectiveness, and computational efficiency.

However, FDPAN still has some weaknesses. 1) In 3WBKHK, the repeated existence of boundary points requires nearest neighbor computation across subsets, which requires more storage space and computing resources. 2) FDPAN will have some shortcomings when dealing with the boundary points of overlapping clusters, decreasing the clustering effect. 3) Although two parameters are positive integers and the interval is bounded, adjusting high-sensitivity parameters causes inconvenience. The shortcomings of FDPAN will still limit its practicality in more complex scenarios, warranting further refinement in future research.

References

[1] R. Ali, T. Farhat, S. Abdullah, and S. Akram, "Customer segmentation for targeted marketing: A comparative analysis of clustering techniques," in *Proc. 25th Int. Multitopic Conf.*, 2023, pp. 1–6.

[2] C. Xu, Z. Jiang, D. Pan, H. Yu, J. Huang, and K. Zhou, "Multiscale neighborhood adaptive clustering image segmentation for molten iron flow slag-iron recognition," *IEEE Trans. Syst. Man Cybern.-Syst.*, vol. 54, no. 8, pp. 4642–4654, Aug. 2024.

[3] A. Li et al., "AngClust: Angle feature-based clustering for short time series gene expression profiles," *IEEE-ACM Trans. Comput. Biol. Bioinform.*, vol. 20, no. 2, pp. 1574–1580, Mar./Apr. 2023.

[4] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowl. Discov. Data Mining*, 1996, pp. 226–231.

[5] A. Rodriguez and A. Laio, "Clustering by fast search and find of density peaks," *Science*, vol. 344, no. 6191, pp. 1492–1496, Jun. 2014.

[6] D. Cheng, Q. Zhu, J. Huang, Q. Wu, and L. Yang, "Clustering with local density peaks-based minimum spanning tree," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 2, pp. 374–387, Feb. 2021.

[7] J. Guan, S. Li, X. Chen, X. He, and J. Chen, "DEMOS: Clustering by pruning a density-boosting cluster tree of density mounts," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 10814–10830, Oct. 2023.

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.

[8] C. Li, S. Ding, X. Xu, H. Hou, and L. Ding, "Fast density peaks clustering algorithm based on improved mutual k-nearest-neighbor and sub-cluster merging," *Inf. Sci.*, vol. 647, Nov. 2023, Art. no. 119470.
[9] M. S. Sarfraz, V. Sharma, and R. Stiefelhagen, "Efficient parameter-free clustering using first neighbor relations," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 8934–8943.
[10] D. Cheng, J. Huang, S. Zhang, X. Zhang, and X. Luo, "A novel approximate spectral clustering algorithm with dense cores and density peaks," *IEEE Trans. Syst. Man Cybern.-Syst.*, vol. 52, no. 4, pp. 2348–2360, Apr. 2022.
[11] T. Qiu and Y. Li, "Fast LDP-MST: An efficient density-peak-based clustering method for large-size datasets," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 4767–4780, May 2023.
[12] D. Cheng, Y. Li, S. Xia, G. Wang, J. Huang, and S. Zhang, "A fast granular-ball-based density peaks clustering algorithm for large-scale data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 12, pp. 17202–17215, Dec. 2024.
[13] S. Ding, W. Du, C. Li, X. Xu, L. Wang, and L. Ding, "Density peaks clustering algorithm based on improved similarity and allocation strategy," *Int. J. Mach. Learn. Cybern.*, vol. 14, no. 4, pp. 1527–1542, Apr. 2023.
[14] R. Liu, H. Wang, and X. Yu, "Shared-nearest-neighbor-based clustering by fast search and find of density peaks," *Inf. Sci.*, vol. 450, pp. 200–226, Jun. 2018.
[15] J. Guan, S. Li, J. Zhu, X. He, and J. Chen, "Fast main density peak clustering within relevant regions via a robust decision graph," *Pattern Recognit*, vol. 152, Aug. 2024, Art. no. 110458.
[16] S. Ding, W. Du, X. Xu, T. Shi, Y. Wang, and C. Li, "An improved density peaks clustering algorithm based on natural neighbor with a merging strategy," *Inf. Sci.*, vol. 624, pp. 252–276, May 2023.
[17] M. Abbas, A. El-Zoghabi, and A. Shoukry, "DenMune: Density peak based clustering using mutual nearest neighbors," *Pattern Recognit.*, vol. 109, Jan. 2021, Art. no. 107589.
[18] W. Zhu, F. Nie, and X. Li, "Fast spectral clustering with efficient large graph construction," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2017, pp. 2492–2496.
[19] S. Ding, C. Li, X. Xu, L. Guo, L. Ding, and X. Wu, "Horizontal federated density peaks clustering," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 820–829, Jan. 2025.
[20] X. Xu, S. Ding, M. Du, and Y. Xue, "DPCG: An efficient density peaks clustering algorithm based on grid," *Int. J. Mach. Learn. Cybern.*, vol. 9, pp. 743–754, 2018.
[21] X. Fang, Z. Xu, H. Ji, B. Wang, and Z. Huang, "A grid-based density peaks clustering algorithm," *IEEE Trans. Ind. Inform.*, vol. 19, no. 4, pp. 5476–5484, Apr. 2023.
[22] S. Ding et al., "A sampling-based density peaks clustering algorithm for large-scale data," *Pattern Recognit.*, vol. 136, Apr. 2023, Art. no. 109238.
[23] L. Ma et al., "UP-DPC: Ultra-scalable parallel density peak clustering," *Inf. Sci.*, vol. 660, Mar. 2024, Art. no. 120114.
[24] X. Xu, S. Ding, Y. Wang, L. Wang, and W. Jia, "A fast density peaks clustering algorithm with sparse search," *Inf. Sci.*, vol. 554, pp. 61–83, Apr. 2021.
[25] J. Lu, Y. Zhao, K.-L. Tan, and Z. Wang, "Distributed density peaks clustering revisited," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 8, pp. 3714–3726, Aug. 2022.
[26] J. Zhu, W. Zhao, H. Yang, and F. Nie, "Joint learning of anchor graph-based fuzzy spectral embedding and fuzzy k-means," *IEEE Trans. Fuzzy Syst.*, vol. 31, no. 11, pp. 4097–4108, Nov. 2023.
[27] F. Nie, W. Zhu, and X. Li, "Unsupervised large graph embedding based on balanced and hierarchical k-means," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 4, pp. 2008–2019, Apr. 2022.
[28] J. Wang, Z. Ma, F. Nie, and X. Li, "Fast self-supervised clustering with anchor graph," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 9, pp. 4199–4212, Sep. 2022.
[29] P. Wang, X. Yang, W. Ding, J. Zhan, and Y. Yao, "Three-way clustering: Foundations, survey and challenges," *Appl. Soft Comput.*, vol. 151, Jan. 2024, Art. no. 111131.
[30] H. Yu, L. Chen, and J. Yao, "A three-way density peak clustering method based on evidence theory," *Knowl.-Based Syst*, vol. 211, Jan. 2021, Art. no. 106532.
[31] P. Wang, T. Wu, and Y. Yao, "A three-way adaptive density peak clustering (3W-ADPC) method," *Appl. Intell.*, vol. 53, no. 20, pp. 23966–23982, Oct. 2023.
[32] H. Ju, Y. Lu, W. Ding, J. Cao, and X. Yang, "Three-way evidence theory-based density peak clustering with the principle of justifiable granularity," *Appl. Soft Comput.*, vol. 152, Feb. 2024, Art. no. 111217.
[33] M. Du, J. Zhao, J. Sun, and Y. Dong, "M3W: Multistep three-way clustering," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 5627–5640, Apr. 2024.
[34] C. Sun, M. Du, J. Sun, K. Li, and Y. Dong, "A three-way clustering method based on improved density peaks algorithm and boundary detection graph," *Int. J. Approx. Reasoning*, vol. 153, pp. 239–257, Feb. 2023.
[35] S. Zhang and K. Li, "A novel density peaks clustering algorithm with isolation kernel and K-induction," *Appl. Sci.-Basel*, vol. 13, no. 1, Jan. 2023, Art. no. 322.
[36] J. Ding, C. Zhang, D. Li, W. Li, and J. Zhan, "A three-way large-scale group decision-making method integrating sentiment analysis and quantum interference-based prospect theory for the selection of new energy vehicles," *Expert Syst. Appl.*, vol. 275, May 2025, Art. no. 126940.
[37] Y. Meng, X. Xu, L. Yan, and Y. Cao, "A three-way density peak clustering algorithm based on sinusoidal fuzzy entropy," *Int. J. Bus. Intell. Data Mining*, vol. 25, no. 2, pp. 184–209, Jan. 2024.
[38] N. Altman and M. Krzywinski, "The curse(s) of dimensionality," *Nat. Methods*, vol. 15, no. 6, pp. 399–400, Jun. 2018.
[39] M. Radovanović, A. Nanopoulos, and M. Ivanović, "Reverse nearest neighbors in unsupervised distance-based outlier detection," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 5, pp. 1369–1382, May 2015.
[40] M. Shekhar, L. Thomas, and K. Karlapalem, "High dimensional clustering: A strongly connected component clustering solution (SCCC)," in *Proc. IEEE Int. Conf. Data Mining Workshops*, 2018, pp. 1104–1111.
[41] C. Li, S. Ding, X. Xu, L. Guo, L. Ding, and X. Wu, "Vertical federated density peaks clustering under nonlinear mapping," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 2, pp. 1004–1017, Feb. 2025.
[42] H. Wang, Y. Yang, and B. Liu, "GMC: Graph-based multi-view clustering," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 6, pp. 1116–1129, Jun. 2020.
[43] J. Guan, S. Li, X. He, and J. Chen, "Clustering by fast detection of main density peaks within a peak digraph," *Inf. Sci.*, vol. 628, pp. 504–521, May 2023.
[44] J. Guan, S. Li, X. He, J. Zhu, J. Chen, and P. Si, "SMMP: A stable-membership-based auto-tuning multi-peak clustering algorithm," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 6307–6319, May 2023.
[45] J. Hou, H. Lin, H. Yuan, and M. Pelillo, "Flexible density peak clustering for real-world data," *Pattern Recognit*, vol. 156, Dec. 2024, Art. no. 110772.
[46] D. Cheng, J. Huang, S. Zhang, S. Xia, G. Wang, and J. Xie, "K-means clustering with natural density peaks for discovering arbitrary-shaped clusters," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 8, pp. 11077–11090, Aug. 2024.

Shifei Ding (Senior Member, IEEE) received PhD degree from the Shandong University of Science and Technology, Qingdao, China, in 2004, and the head of Joint Laboratory of Intelligent Information Processing, China University of Mining and Technology and Chinese Academy of Sciences. He is a CAAI fellow. His research interests include intelligent information processing, pattern recognition, machine learning, data mining, and granular computing et al.

Chao Li received the BS degree from Henan Polytechnic University, Jiaozuo, China, in 2020. He is currently working toward the PhD degree with the School of Computer Science and Technology, China University of Mining and Technology. His current research interests include machine learning, and clustering analysis.

Xiao Xu, photograph and biography not available at the time of publication.

Lili Guo, (Member, IEEE) photograph and biography not available at the time of publication.

Ling Ding, photograph and biography not available at the time of publication.

Xindong Wu, (Fellow, IEEE) photograph and biography not available at the time of publication

Authorized licensed use limited to: CHINA UNIVERSITY OF MINING AND TECHNOLOGY. Downloaded on April 16,2026 at 01:43:17 UTC from IEEE Xplore. Restrictions apply.